

UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE COLOMBIA

Ingeniería de Sistemas y Computación

Programación 3

Ejercicio de práctica - Sockets (cliente envía, servidor transforma)

Versión

1.0

Propósito

Practicar el uso de **sockets TCP** y **DataStream/DataOutputStream** en Java para:

1. Enviar un mensaje desde un **cliente**.
2. Transformar el mensaje a **mayúsculas** en el **servidor**.
3. Devolver el resultado al cliente.
4. El servidor debe mostrar en consola:
 - El mensaje recibido.
 - El mensaje enviado (transformado).

Enunciado

Crear un programa cliente-servidor en Java donde:

1. Existe **un solo servidor** que escucha en un puerto TCP.
2. Un **cliente** se conecta al servidor y envía un mensaje de texto.
3. El servidor debe:
 - Recibir el mensaje del cliente.
 - Convertirlo a **mayúsculas**.
 - Imprimir en pantalla el mensaje recibido y el mensaje enviado.
 - Enviar de vuelta el mensaje transformado al cliente.
4. El cliente debe imprimir el mensaje recibido desde el servidor.

Entradas (parámetros)

Servidor: UpperServer

Ejecutar con: `java UpperServer <port>`

Ejemplo: `java UpperServer 5000`

Cliente: UpperClient

Ejecutar con: `java UpperClient <host> <port> <mensaje>`

Ejemplo: `java UpperClient localhost 5000 "hola mundo"`

Protocolo de comunicación (obligatorio)

Usar `DataInputStream` y `DataOutputStream`.

Cliente → Servidor

1. Envía `int msgLen` (longitud del mensaje en bytes UTF-8)
2. Envía `byte[msgLen]` con el mensaje en UTF-8

Servidor → Cliente

1. Envía `int msgLen` (longitud del mensaje transformado en bytes UTF-8)
2. Envía `byte[msgLen]` con el mensaje transformado en UTF-8

Reglas (obligatorias)

1. El servidor debe transformar el mensaje con:
 - `mensaje.toUpperCase(Locale.ROOT)`
2. El servidor debe imprimir en consola (en cada solicitud):
 - Recibido: `<mensaje>`
 - Enviado: `<mensajeTransformado>`
3. El mensaje debe devolverse al mismo cliente que lo envió.

Requerimientos funcionales

1. El servidor debe:
 - Crear un `ServerSocket`.
 - Aceptar conexión del cliente.
 - Leer el mensaje con el protocolo indicado.
 - Convertir a mayúsculas.
 - Enviar el resultado usando el protocolo indicado.
2. El cliente debe:
 - Conectarse al servidor.

- Enviar el mensaje con el protocolo indicado.
- Leer la respuesta y mostrarla en consola.

Criterios de aceptación

1. Si el cliente envía `hola mundo` , el cliente recibe `HOLA MUNDO` .
2. El servidor imprime correctamente el mensaje recibido y el mensaje enviado.
3. El servidor no debe finalizar inesperadamente ante una desconexión abrupta del cliente.

Ejemplos de entrada y salida

Ejemplo 1 (mensaje simple)

Entrada (cliente): `java UpperClient localhost 5000 "hola mundo"`

Salida esperada (servidor): `Recibido: hola mundo Enviado: HOLA MUNDO`

Salida esperada (cliente): `Recibido del servidor: HOLA MUNDO`

Ejemplo 2 (incluye espacios)

Entrada (cliente): `java UpperClient localhost 5000 "programacion 3"`

Salida esperada (servidor): `Recibido: programacion 3 Enviado: PROGRAMACION 3`

Salida esperada (cliente): `Recibido del servidor: PROGRAMACION 3`

Entregables

1. Código fuente en Java:
 - `UpperServer.java`
 - `UpperClient.java`